



中山大學
SUN YAT-SEN UNIVERSITY

继承和派生

中山大学计算机学院

主讲人：郑贵锋

zhenggf@mail.sysu.edu.cn



中山大学MOOC课程组

目录

CONTENTS

01

继承/派生的概念

02

语法与访问控制

03

构造与析构顺序

04

派生与构造函数

05

派生与成员函数

06

改变访问控制

07

类型兼容性

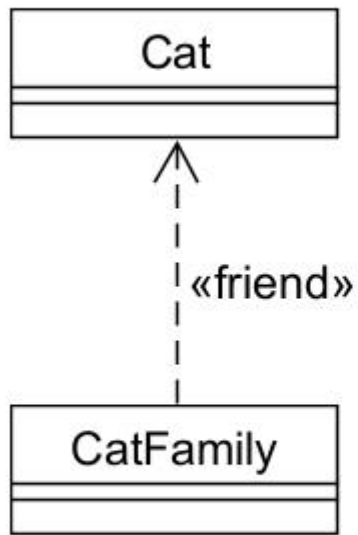
08

类的类型转换

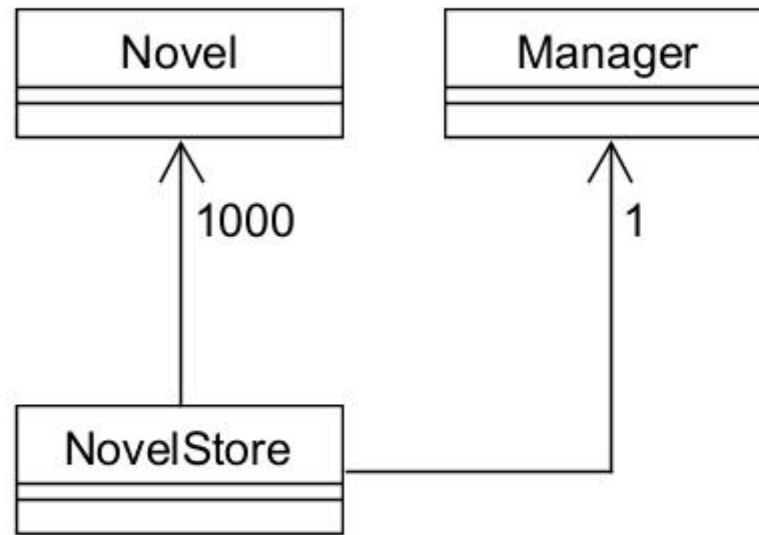
09

多重继承

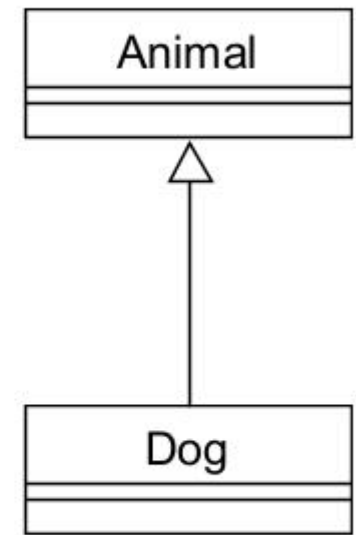
对象之间的关系与面向对象编程 (OOP)



友元关系



组合关系



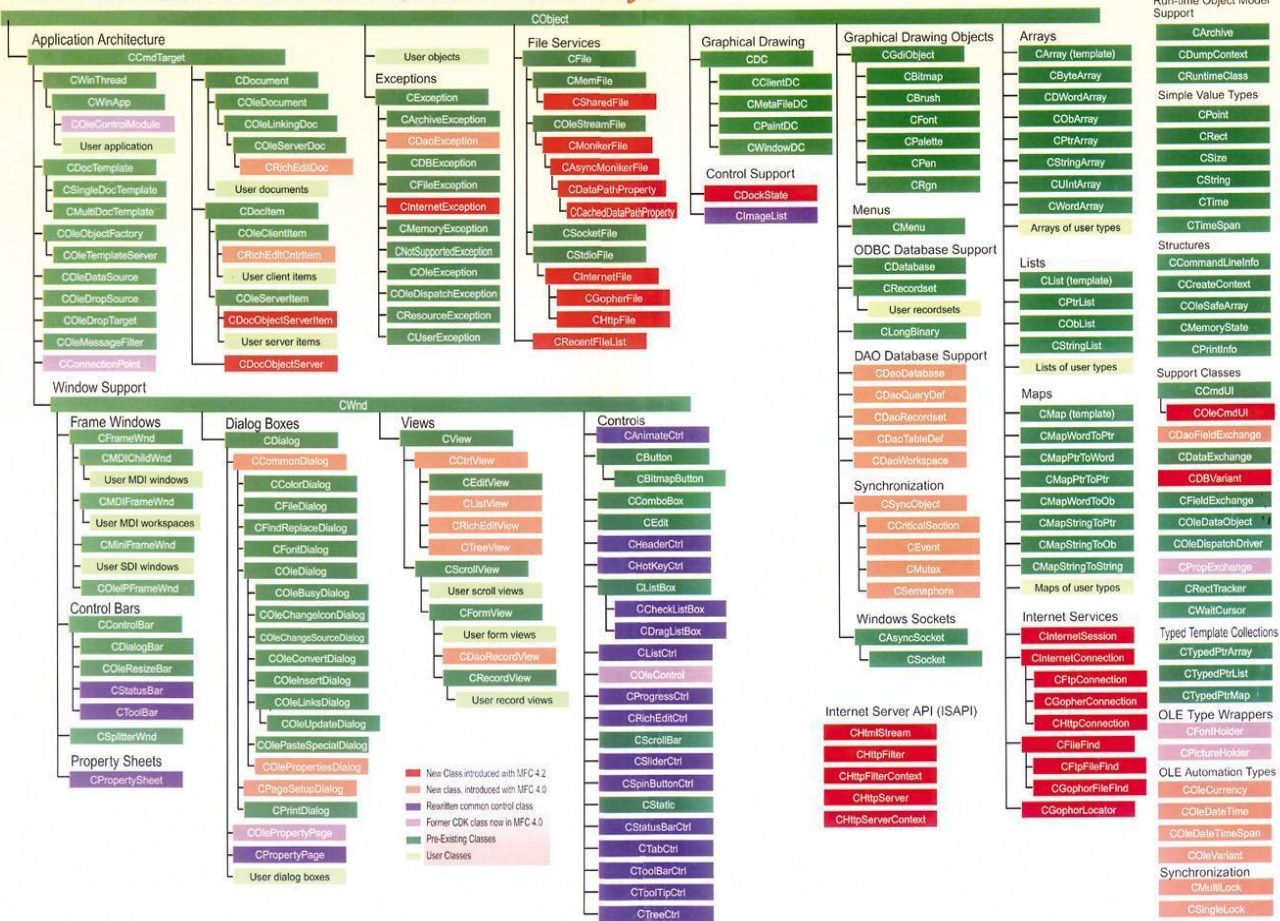
继承关系

理解对象之间关系与OOP的重要性

是我们理解、开发大型程序或应用的基础

面向对象的程序和现实世界一样，都是在已有事物基础上，不断优化组合，继承、变异，演化形成新的事物和应用

MFC 4.21 类别组织框架图 (Class Hierarchy)





继承与派生的相关概念

- 在 C++ 中，当定义一个新的类 B 时，如果发现类 B 拥有某个已写好的类 A 的全部特点，此外还有类 A 没有的特点，那么就不必从头重写类 B，而是可以把类 A 作为一个“**基类**”（也称“**父类**”），把类 B 写为基类 A 的一个“**派生类**”（也称“**子类**”）。这样，就可以说从类 A “**派生**”出了类 B，也可以说类 B “**继承**”了类 A。
- 派生类是通过对基类进行扩充和修改得到的。基类的所有成员自动成为派生类的成员。

例子——学生与毕业生

Student

Base class (基类) ②
Parent class (父类)
Super class (超类)

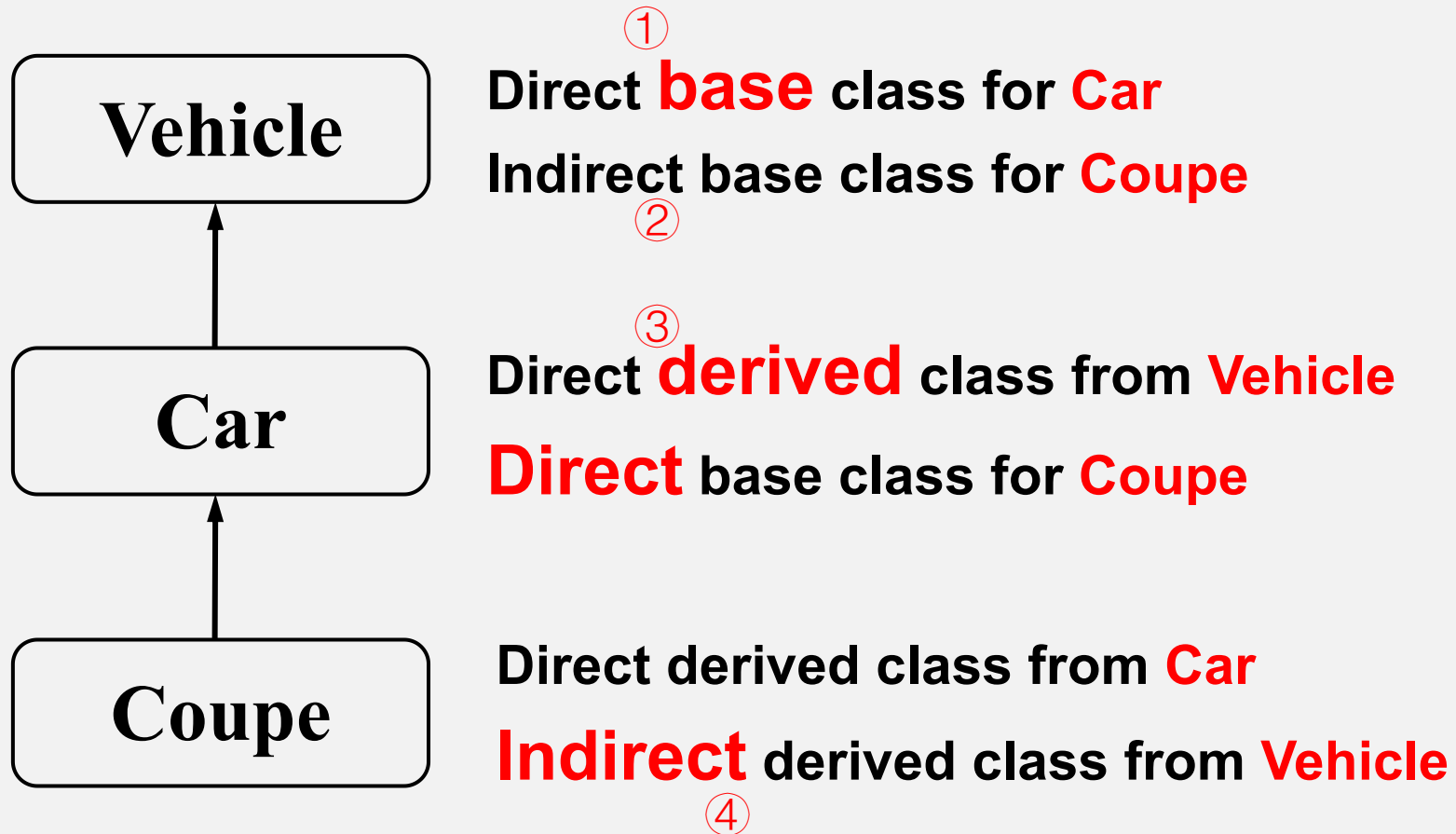
GradStudent

Derived class (派生类) ③
Sub class (子类)

① 继承(Inheritance)代表了从属(is-a)关系
④ GradStudent 属于(is a) student

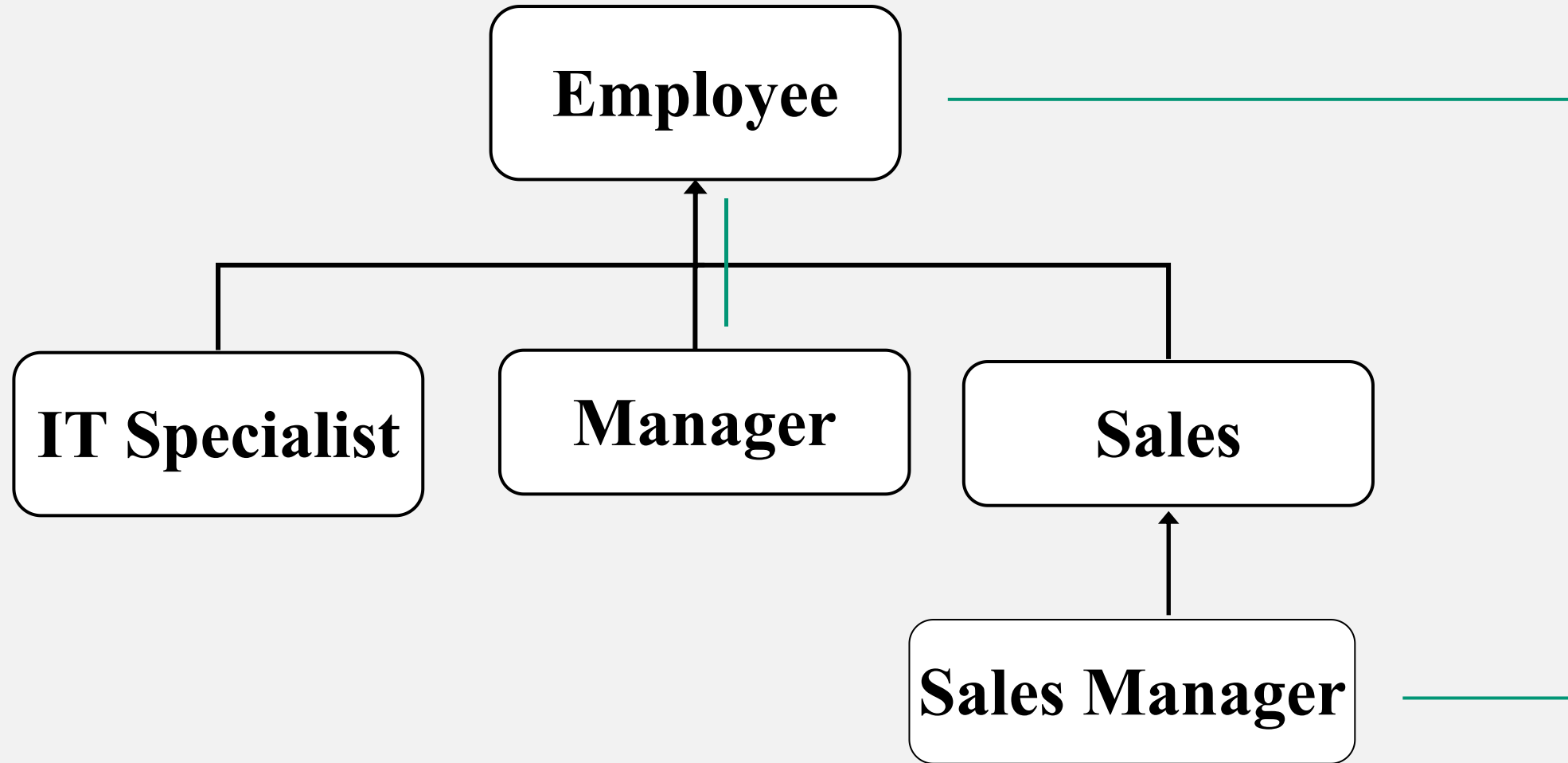


例子——Vehicle、Car、Coupe





例子——职业分类



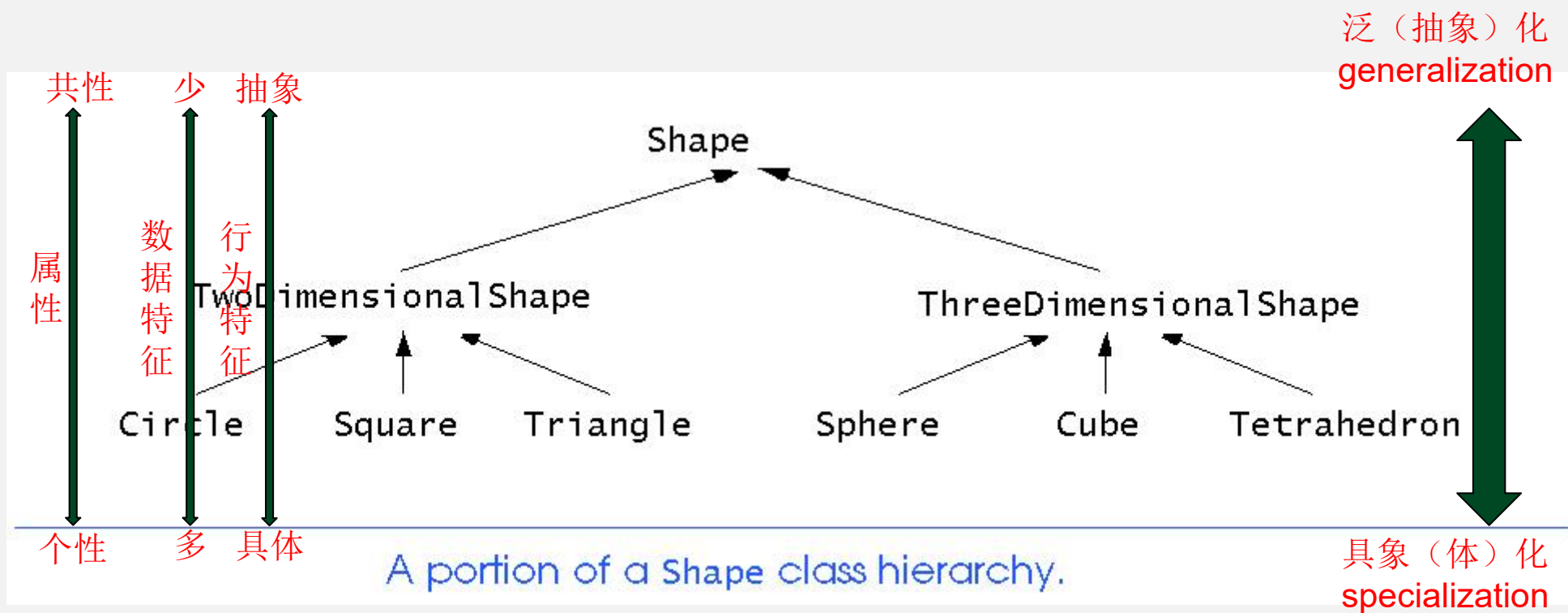


为什么要提出继承与派生的概念？

- 提供派生类的概念及其相关语言机制是为了表达层次关系，即表达类之间的**共性**。派生类是通过对基类进行**扩充**和**修改**得到的。**基类的所有成员自动成为派生类的成员**。
- 它是通常所说的**面向对象编程**的基础。
- 如果只有一个类的概念，软件的可重用性、演化和相关的概念表示存在严重的不灵活问题。继承机制为软件可重用性、IS-A 概念表示和易于修改提供了解决方案。
- 继承提供了一种通过修改（演化）一个或多个现有类来构造新类的方法。



继承与派生的优势



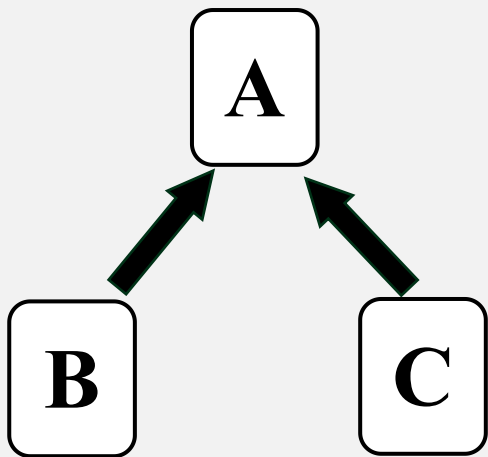


继承与派生概念小结

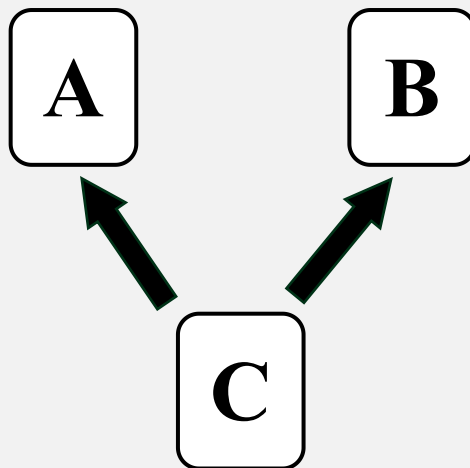
- 使得程序可刻画现实世界的IS-A关系。
- 提高程序的可重用性
 - 派生类重用基类类的代码可提高程序开发效率。派生类的定义通常基于设计完善、并经严格测试的基类，从而使程序设计工作建立在一个可靠的基础上，有助于高效地开发出可靠性较高的软件
 - 这种重用是一种灵活的重用方式：子类在继承父类代码的基础上，可根据自己的特性进行调整。
- 定义：“类B继承类A”或“类A派生类B”

在类B中除了自己定义的成员之外，还自动包括了类A中定义的数据成员与成员函数，这些自动继承下来的成员称为类B的继承成员

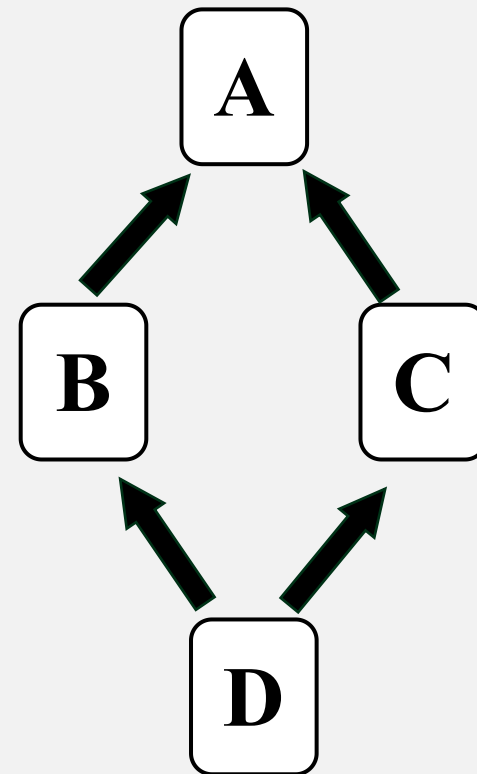
C++中的继承关系图解



派生类只有一个直接基类
(单继承)



派生类具有两个及以上直接基类
(多重继承)

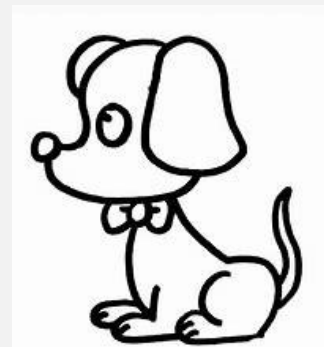


多重继承的一种特殊形式：派生类两次或两次以上重复继承某个祖先类

C++ 继承关系语法基础

```
class Animal {  
    public:  
        void eat() {  
            isEaten = true;  
            cout << "I have eaten!" << endl;  
        };  
    → protected:  
        Animal() {cout << "Animal is constructing" << endl;};  
        bool isEaten = false;  
};
```

```
class Dog:public Animal {  
    public:  
        Dog():Animal() {cout << "Dog is constructing" << endl;};  
        void bark(){  
            if (isEaten) ←  
                cout << "I am Barking, wang wang wang" << endl;  
            }  
            else {  
                cout << "I am starving, wow" << endl;  
            };  
        };  
};
```





C++ 继承关系语法基础

```
int main() {  
    Dog dog;  
    dog.eat(); ←  
    dog.bark();  
}
```

\$./a

Animal is constructing

Dog is constructing

I have eaten!

I am Barking, wang wang wang

要点:

- ① public, **protected**, private
- ② class Derived:**public Super**
- ③ 构造函数, 新成员数据, 函数

假设有类 Cat, 有一个新成员函数 catchMice, 它吃饱了就抓老鼠, 否则喵喵叫。

代码复用!



继承的语法

- 单重继承的定义形式

```
class 派生类名: 继承访问控制 基类类名{  
    成员访问控制:  
        成员声明列表;  
};
```

- 继承访问控制和成员访问控制均由保留 `public`、`protected`、`private` 来定义，缺省均为 `private`。

继承中的访问控制

- **private**（私有的）：

在private后声明的成员称为私有成员，私有成员只能通过本类的成员函数来访问。

- **public**（公有的）：

在public后声明的成员称为公有成员，公有成员用于描述一个类与外部世界的接口，类的外部（程序的其它部分的代码）可以访问公有成员。

- **protected**（受保护的）：

受保护成员具有private与public的双重角色：对派生类的成员函数而言，它为public，而对类的外部而言，它为private。即：protected成员只能由本类及其后代类的成员函数访问。



继承中的访问控制

- 影响继承成员（派生类从基类中继承而来的成员）访问控制方式的两个因素：
 - 定义派生类时指定的继承访问控制
 - 该成员在基类中所具有的成员访问控制

class B: 继承访问控制 A {

成员访问控制:

成员声明列表;

} ;



继承中的访问控制

基类中成员的访问控制	继承访问控制	派生类中继承成员的访问控制
public	public	public
protected		protected
private		不可访问
public	protected	protected
protected		protected
private		不可访问
public	private	private
protected		private
private		不可访问

- 无论采用什么继承方式，基类的私有成员在派生类中都是不可访问的。
- “私有”和“不可访问”有区别：私有成员可以由派生类本身访问，不可访问成员即使是派生类本身也不能访问。
- 大多数情况下均使用**public**继承



访问控制举例

```
class BASE
{
    public:
        BASE();
        void get_ij();
    protected:
        int i, j;
    private:
        int x_temp;
};
```

//公有派生：在Y1类中，i、j是受保护成员

```
class Y1:public BASE
{
    public:
        void increment();    //get_ij()是公有成员，x_temp不可访问
    private:
        float nmember;
};
```



访问控制举例

```
BASE::BASE()
{ i=0; j=0; x_temp=0; }

void BASE:: get_ij()
{
    cout << i << ' ' <<
j << endl;
}

void Y1::increment()
{
    i++; j++;
}
```

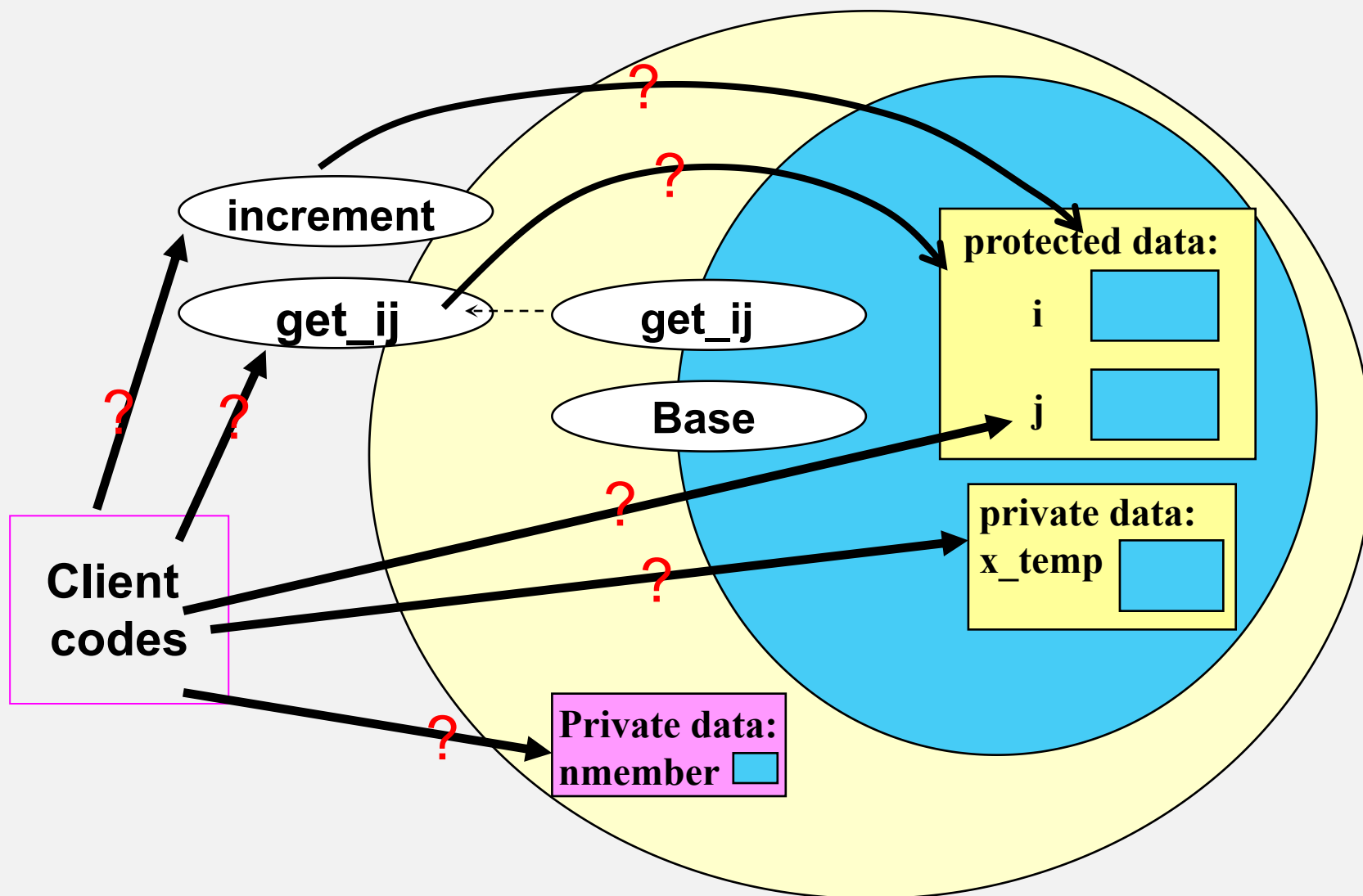
```
int main() //程序Access
{
    BASE obj1;
    Y1 obj2;

    obj2.increment();
    obj2.get_ij();
    obj1.get_ij();
}
```

运行程序 屏幕显示:

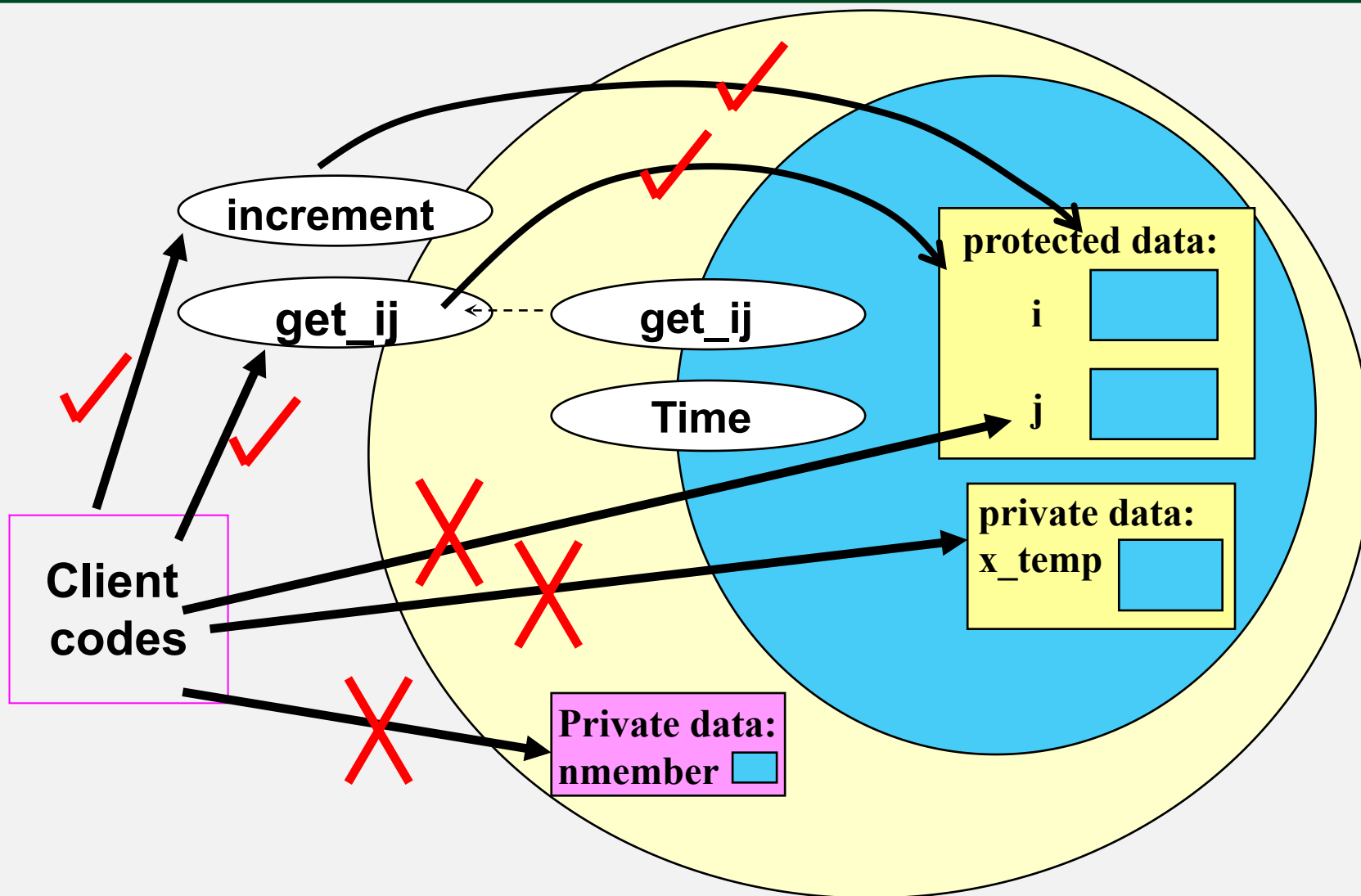
```
1 1
0 0
```

访问控制举例分析



main函数能访问到哪些成员函数与成员变量?

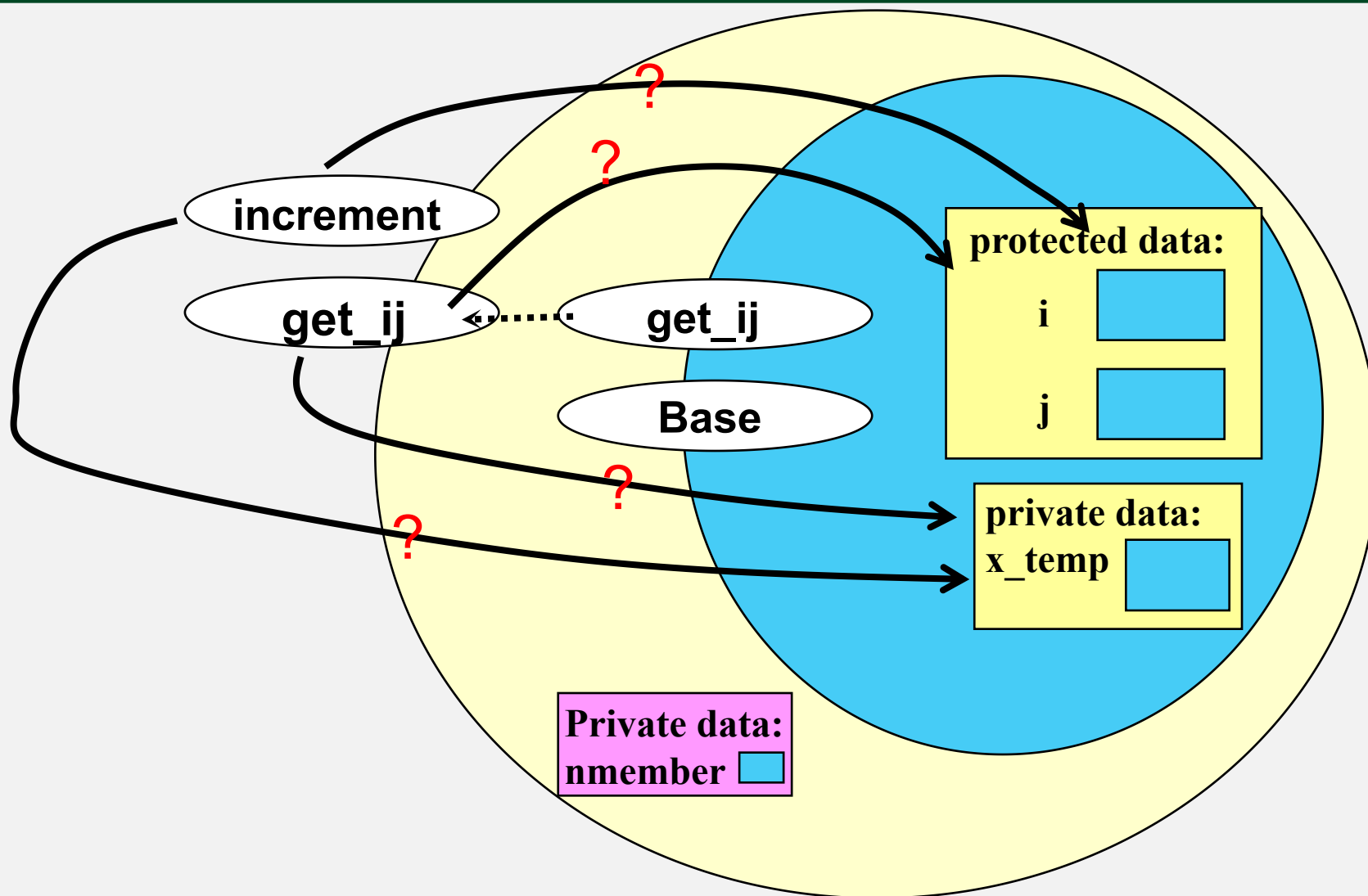
访问控制举例分析



main函数能访问到哪些成员函数与成员变量？



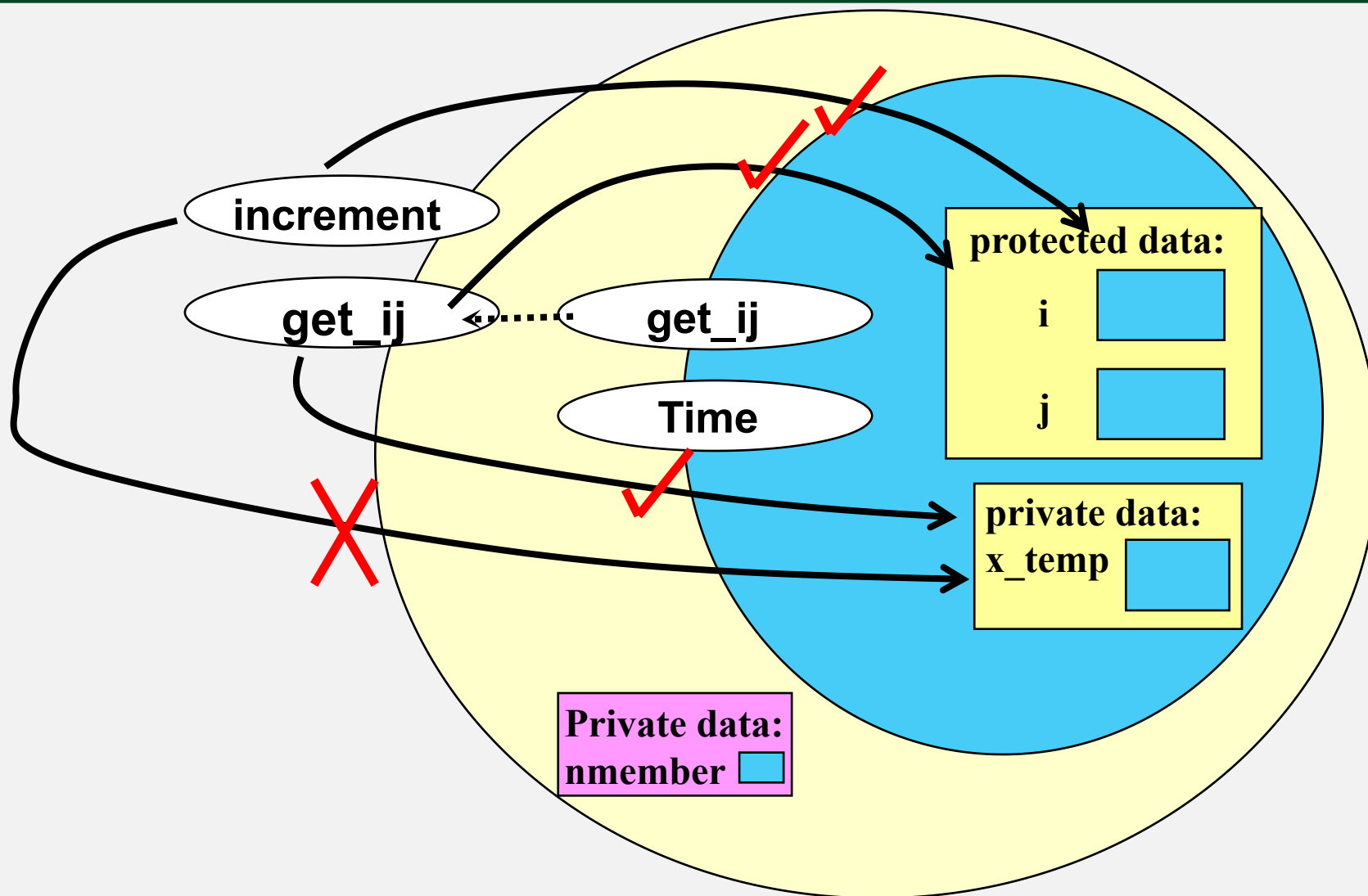
访问控制举例分析



派生类能够访问
哪些对象？



访问控制举例分析



派生类能够访问
哪些对象？



访问控制小结

```
class BASE{  
    protected:    int i, j;  
    public:       void get_ij();  
    private:      int x_temp;  
};
```

保护派生：在Y2类中，i、j是受保护成员。get_ij()变成受保护成员，x_temp不可访问

```
class Y2:protected BASE{ ... };
```

私有派生：在Y3类中，i、j、get_ij()都变成私有成员，x_temp不可访问

```
class Y3:private BASE{ ... };
```



继承成员的访问控制(Tip)

- 在大多数情况下，使用**public**的继承方式；**private**和**protected**是很少使用的。
- ✓ 微软的**MFC**：全部使用**public**的继承方式
- ✓ **AT&T**的**iostream**库：95%以上使用的是**public**的继承方式



派生类对象的存储

- 派生类的对象不仅存放了在派生类中定义的非静态数据成员，而且也存放了从基类中继承下来的非静态数据成员
- 可以认为派生类对象中包含了基类子对象。

BASE obj1;

Y1 obj2;

obj1

i
j
x_temp

obj2

i
j
x_temp
nmember



继承时的构造函数

- 基类的构造函数**不被继承**，派生类中需要声明自己的构造函数。
- 派生类的构造函数中只需要对本类中新增成员进行初始化即可。
对继承来的基类成员的初始化是通过编译在派生类构造函数初始化器中自动生成默认构造函数（默认拷贝构造）完成。
 - 如果基类没有默认构造（包括 `=delete`），**则编译错误**
- 派生类的构造函数需要使用基类的有参构造函数，必须显式在初始化器列表中申明。**注意：不能在构造函数内调用！**



继承时的构造函数

- 构造函数的调用次序（创建派生类对象时）
 - 首先调用其基类的构造函数（调用顺序按照基类被继承时的声明顺序（从左向右））。
 - 然后调用本类对象成员的构造函数（调用顺序按照对象成员在类中的声明顺序）。
 - 最后调用本类的构造函数。



继承时的析构函数

- 撤销派生类对象时析构函数的调用次序与构造函数的调用次序相反
 - 首先调用本类的析构函数
 - 然后调用本类对象成员的析构函数
 - 最后调用其基类的析构函数



继承时的构造函数、析构函数举例

//Demo.h

```
class C {  
public:  
    C( );    //构造函数  
    ~C( );   //析构函数  
};  
  
class BASE {  
public:  
    BASE( );    // 构造函数  
    ~BASE( );   // 析构函数  
};
```

#include "Demo.h"

//Demo.cpp

```
C::C( )    //构造函数  
{    cout << "Constructing C object.\n"; }  
  
C::~~C( )  //析构函数  
{    cout << "Destructing C object.\n"; }  
  
BASE::BASE( )    // 构造函数  
{    cout << "Constructing BASE object.\n"; }  
  
BASE::~~BASE( )  // 析构函数  
{    cout << "Destructing BASE object.\n";  
    }
```



继承时的构造函数、析构函数举例

```
class DERIVED: public BASE {    // Derived.h
public:
    DERIVED()                  // 构造函数
    ~DERIVED()                 // 析构函数
private:
    C mOBJ;
};
```

```
#include "Derived.h"    // Client.cpp

int main()
{
    DERIVED obj;    // 声明一个派生类的对象
                  // 什么也不做, 仅完成对象obj的构造与析构

    return 0;
}
```

```
#include "Derived.h"    // Derived.cpp

DERIVED::DERIVED()      // 构造函数
{    cout << "Constructing derived object.\n";    }

DERIVED::~~DERIVED()    // 析构函数
{    cout << "Destructing derived object.\n";    }
```

运行结果:

Constructing BASE object.

Constructing C object.

Constructing derived object.

Destructing derived object.

Destructing C object.

Destructing BASE object.